

Deep Dive — Module B

LangGraph — Complete Documentation

From LangChain basics to multi-agent systems and production deployment

ContentAutomation2 — Backend Deep-Dive Series
Complete Documentation | Zero Prerequisites to Production Code

Table of Contents

1. What is LangChain? What is LangGraph?
2. Messages — The Communication Protocol
3. Chat Models — Talking to LLMs
4. Prompt Templates
5. Tools — What Agents Can Do
6. Tool Calling — How the LLM Uses Tools
7. State — The Agent's Memory
8. StateGraph — Building the Graph
9. Nodes — Units of Work
10. Edges — Connecting Nodes
11. Conditional Edges — Branching Logic
12. Compiling and Running Graphs
13. Streaming — Watching the Agent Think
14. Checkpointers — Persistence and Memory
15. Human-in-the-Loop — Interrupts and Approvals
16. Prebuilt ReAct Agent — Fast Start
17. ToolNode — Automatic Tool Execution
18. Multi-Agent Systems — Supervisor Pattern
19. Subgraphs — Modular Architectures
20. Error Handling in Graphs
21. LangGraph + FastAPI — Full Integration
22. ContentAutomation2 — Orchestrator Walkthrough
23. Testing LangGraph Agents
24. Debugging & Tracing with LangSmith

1. What is LangChain? What is LangGraph?

LangChain is a Python framework for building LLM-powered applications. It provides abstractions for: chat models, prompts, tools, output parsers, document loaders, vector stores, and memory. Think of it as the standard library for LLM applications.

LangGraph is built on top of LangChain. It adds structured, stateful workflows. Instead of just calling an LLM in a linear chain, LangGraph lets you build cyclic graphs where the LLM can reason, call tools, observe results, and decide what to do next — an indefinite number of times.

The key insight: an **agent** is a loop. LangGraph makes that loop explicit, controllable, and observable.

```
pip install langgraph langchain langchain-anthropic

# LangGraph core concepts:
# - StateGraph: the graph itself
# - State: a dict that persists across all nodes
# - Node: a Python function that transforms the state
# - Edge: a connection from one node to another
# - Conditional edge: routes to different nodes based on state
# - Checkpointer: saves state so the graph can be resumed
```

2. Messages — The Communication Protocol

LangGraph agents communicate using a list of **messages**. Every message has a role (who sent it) and content (what was said).

```
from langchain_core.messages import (
    HumanMessage,      # from the user
    AIMessage,          # from the LLM
    SystemMessage,      # system instructions
    ToolMessage,        # result from a tool call
)

# Create messages
human = HumanMessage(content="What is the repo interest rate?")
system = SystemMessage(content="You are an expert in Indian finance.")
ai = AIMessage(content="The repo rate is currently 6.50%.")

# ToolMessage – returned after executing a tool
tool_result = ToolMessage(
    content='{"rate": 6.50, "date": "2026-06-01"}',
    tool_call_id="call_abc123",  # must match the ID from AIMessage
)

# AIMessage with tool_calls – LLM is requesting a tool
ai_with_tool = AIMessage(
    content="",  # empty when requesting a tool
    tool_calls=[{
        "id": "call_abc123",
        "name": "get_rbi_rate",
        "args": {"date": "latest"},
    }]
)

# Full conversation flow:
```

```

messages = [
    SystemMessage(content="You help with Indian finance questions."),
    HumanMessage(content="What's the repo rate?"),
    ai_with_tool,          # LLM decides to call get_rbi_rate
    tool_result,          # tool result injected back
    AIMessage(content="The repo rate is 6.50%."), # LLM final answer
]

```

3. Chat Models — Talking to LLMs

```

from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage, SystemMessage

# Create the model
llm = ChatAnthropic(
    model="claude-sonnet-4-6",
    api_key="YOUR_API_KEY",
    max_tokens=4096,
    temperature=0,          # 0 = deterministic, 1 = creative
)

# Simple call – invoke with a list of messages
response = llm.invoke([
    SystemMessage(content="Be concise."),
    HumanMessage(content="What is SEBI?"),
])
print(response.content)      # "SEBI is Securities..."
print(type(response))        # AIMessage

# Async call
response = await llm.ainvoke([HumanMessage(content="Hello")])

# Streaming
for chunk in llm.stream([HumanMessage(content="Tell me about RBI")]):
    print(chunk.content, end="", flush=True)

# Async streaming
async for chunk in llm.astream([HumanMessage(content="Tell me about RBI")]):
    print(chunk.content, end="", flush=True)

# Batch calls (multiple prompts at once)
responses = llm.batch([
    [HumanMessage(content="Question 1")],
    [HumanMessage(content="Question 2")],
])

```

4. Prompt Templates

```

from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

# ■■■ Static template ■■■
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert in {domain}. Be concise."),
    ("human", "Explain {concept} in simple terms."),
])

# Render with values

```

```

messages = prompt.format_messages(domain="Indian finance", concept="repo rate")

# ■■■ With conversation history placeholder ■■■
prompt = ChatPromptTemplate.from_messages([
    ("system", "You help users understand {topic}.",
    MessagesPlaceholder("history"), # inject full message history here
    ("human", "{input}"),
])

messages = prompt.format_messages(
    topic="Indian stocks",
    history=[HumanMessage("Tell me about NIFTY"), AIMessage("NIFTY is...")],
    input="What drove the rally today?",
)

# ■■■ Chaining prompt → model ■■■
chain = prompt | llm
response = chain.invoke({
    "topic": "Indian stocks",
    "history": [],
    "input": "What is the NIFTY 50?",
})

```

5. Tools — What Agents Can Do

A tool is a Python function that the LLM can decide to call. The LLM sees the tool's name and description (from the docstring) and chooses when to use it.

```

from langchain_core.tools import tool
from typing import Optional

# ■■■ @tool decorator – simplest way to create a tool ■■■
@tool
def get_stock_price(ticker: str) -> str:
    """Get the current stock price for an Indian stock ticker.
    ticker: NSE ticker symbol (e.g. 'RELIANCE', 'INFY', 'TCS')"""
    # Your actual API call here
    return f"{ticker}: INR 2450.75"

# LangChain reads:
print(get_stock_price.name)           # "get_stock_price"
print(get_stock_price.description)    # the docstring
print(get_stock_price.args_schema)    # {"ticker": {"type": "string", ...}}

# ■■■ Tool with multiple parameters ■■■
@tool
def search_ideas(
    status: str = "pending_approval",
    platform: Optional[str] = None,
    limit: int = 10,
) -> str:
    """Browse content ideas from the database.
    status: 'pending_approval', 'approved', or 'rejected'.
    platform: filter by 'linkedin', 'twitter', 'blog', or 'email'.
    limit: max number of results (default 10)."""
    results = db.table("ideas").select("*").eq("approval_status", status).limit(limit).execute()
    return str(results.data)

# ■■■ Async tool ■■■

```

```

@tool
async def trigger_agent(agent_name: str) -> str:
    """Trigger an agent task in the background.
    agent_name: 'research', 'scoring', or 'creation'"""
    job = await arq_pool.enqueue_job(f"{agent_name}_agent_task")
    return f"Triggered {agent_name} (job_id={job.job_id})"

# ■■■ Tool with pydantic schema (explicit, more reliable) ■■■
from pydantic import BaseModel, Field
from langchain_core.tools import StructuredTool

class StockInput(BaseModel):
    ticker: str = Field(description="NSE ticker symbol")
    exchange: str = Field(default="NSE", description="NSE or BSE")

def get_price(ticker: str, exchange: str = "NSE") -> str:
    return f"{exchange}:{ticker} = INR 2450.75"

stock_tool = StructuredTool.from_function(
    func=get_price,
    name="get_stock_price",
    description="Get the current stock price",
    args_schema=StockInput,
)

```

5.1 Tool Closures — Injecting Shared State

Problem: tools need a database client, but can't accept it as an LLM arg
 # Solution: define tools INSIDE a factory function (closure)

```

from supabase import Client
from arq import ArqRedis

```

```

def make_tools(supabase: Client, arq_pool: ArqRedis) -> list:

```

```

    @tool
    async def get_ideas(status: str = "pending_approval") -> str:
        """Browse content ideas. status: pending_approval, approved, rejected."""
        # 'supabase' is captured from the outer scope – this is a closure!
        resp = supabase.table("ideas").select("*").eq("approval_status", status).execute()
        ideas = resp.data or []
        lines = [f"[{i['platform']}] score={i.get('score','?')} | [{i['angle']}]" for i in ideas]
        return "\n".join(lines) if lines else "No ideas found."

```

```

    @tool
    async def approve_idea(idea_id: str, edited_angle: Optional[str] = None) -> str:
        """Approve a content idea at Gate 1.
        idea_id: UUID from get_ideas. edited_angle: optional revised angle."""
        payload = {"approval_status": "approved"}
        if edited_angle:
            payload["edited_angle"] = edited_angle
        resp = supabase.table("ideas").update(payload).eq("id", idea_id).execute()
        return f"Approved idea {idea_id[:8]}..."

```

```

    @tool
    async def run_research(topic: Optional[str] = None) -> str:
        """Trigger the research agent to scrape news sites."""
        # 'arq_pool' is also captured from the outer scope
        job = await arq_pool.enqueue_job("research_agent_task", topic=topic)
        return f"Research started (job_id={job.job_id})"

```

```
        return [get_ideas, approve_idea, run_research]

# At startup:
tools = make_tools(supabase_client, arq_pool_instance)
```

6. Tool Calling — How the LLM Uses Tools

```
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage

llm = ChatAnthropic(model="claude-sonnet-4-6")
tools = [get_stock_price, search_ideas, approve_idea]

# Bind tools to the model – the LLM now knows they exist
llm_with_tools = llm.bind_tools(tools)

# Invoke – the LLM may or may not call a tool
response = await llm_with_tools.ainvoke([
    HumanMessage("Show me pending LinkedIn ideas with score above 7")
])

# Check if the LLM wants to call a tool
if response.tool_calls:
    for call in response.tool_calls:
        print(f"Tool: {call['name']}")
        print(f"Args: {call['args']}")
        print(f"ID: {call['id']}")
else:
    # LLM answered directly without needing a tool
    print(f"Answer: {response.content}")
```

6.1 Manually Executing Tool Calls

```
from langchain_core.messages import ToolMessage

# Build a tool lookup dict
tool_map = {t.name: t for t in tools}

async def execute_tool_calls(ai_message: AIMessage) -> list[ToolMessage]:
    results = []
    for call in ai_message.tool_calls:
        tool = tool_map[call["name"]]
        # Invoke the tool with the args the LLM chose
        if asyncio.iscoroutinefunction(tool.func):
            result = await tool.ainvoke(call["args"])
        else:
            result = tool.invoke(call["args"])
        results.append(ToolMessage(
            content=str(result),
            tool_call_id=call["id"], # must match the call ID
        ))
    return results

# Simple manual agent loop:
messages = [HumanMessage("Show pending ideas")]
for _ in range(10): # max 10 iterations
    response = await llm_with_tools.ainvoke(messages)
    messages.append(response)
```

```

if not response.tool_calls:
    print("Final answer:", response.content)
    break

tool_results = await execute_tool_calls(response)
messages.extend(tool_results)

```

7. State — The Agent's Memory

State is the data that persists across all nodes in the graph. Every node receives the current state and returns an update to it. LangGraph merges the update into the state using the reducer functions you define.

```

from typing import TypedDict, Annotated, Sequence
from langchain_core.messages import BaseMessage
import operator

# ■■ Basic state with message accumulation ■■
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], operator.add]
    # operator.add means: new messages are ADDED to existing ones
    # (not replaced) – this is the standard pattern

# ■■ Extended state with more fields ■■
class OrchestratorState(TypedDict):
    messages: Annotated[list[BaseMessage], operator.add]
    user_id: str          # who is chatting
    session: str          # conversation session ID
    context: dict         # extra context passed around

# ■■ Custom reducer (replace instead of add) ■■
def last_item_reducer(current, update):
    return update  # always replace with latest

class MyState(TypedDict):
    messages: Annotated[list, operator.add]  # accumulate
    last_result: Annotated[dict, last_item_reducer]  # overwrite

# ■■ Reading and updating state in a node ■■
async def my_node(state: AgentState) -> dict:
    messages = state["messages"]  # read current messages
    # Return ONLY the keys you want to update
    # LangGraph merges this with the existing state
    return {"messages": [AIMessage(content="I processed your request")]}

```

8. StateGraph — Building the Graph

```

from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
from langchain_core.messages import BaseMessage
import operator

# Define state
class AgentState(TypedDict):
    messages: Annotated[list[BaseMessage], operator.add]

# Create graph

```

```

graph = StateGraph(AgentState)

# Add nodes (functions)
graph.add_node("llm_node", call_llm_function)
graph.add_node("tool_node", execute_tools_function)

# Set entry point – where execution starts
graph.set_entry_point("llm_node")

# Add edges
graph.add_edge("tool_node", "llm_node") # after tools, go back to LLM

# Add conditional edge – routing based on state
graph.add_conditional_edges(
    "llm_node",          # from this node
    route_function,      # call this to decide next step
    {
        "use_tool": "tool_node", # route name → node name
        "end":      END,
    }
)

# Compile – validates the graph and returns a runnable
agent = graph.compile()

```

9. Nodes — Units of Work

A node is any Python function (sync or async) that takes the state dict and returns a dict of updates. The function name becomes the node name.

```

from langchain_anthropic import ChatAnthropic
from langchain_core.messages import AIMessage

llm = ChatAnthropic(model="claude-sonnet-4-6")
tools = make_tools(supabase, arq_pool)
llm_with_tools = llm.bind_tools(tools)

# ■■■ Node 1: LLM node ■■■
async def call_llm(state: AgentState) -> dict:
    messages = state["messages"]
    response = await llm_with_tools.ainvoke(messages)
    return {"messages": [response]} # add AI response to history

# ■■■ Node 2: Tool execution node ■■■
from langchain_core.messages import ToolMessage

async def call_tools(state: AgentState) -> dict:
    last_message = state["messages"][-1] # the AIMessage requesting tools
    tool_map = {t.name: t for t in tools}
    results = []
    for call in last_message.tool_calls:
        tool = tool_map[call["name"]]
        try:
            result = await tool.ainvoke(call["args"])
        except Exception as e:
            result = f"Error: {e}"
        results.append(ToolMessage(
            content=str(result),

```

```

        tool_call_id=call["id"],
    ))
    return {"messages": results}

# ■■ Node 3: A simple transformation node ■■
def format_output(state: AgentState) -> dict:
    last = state["messages"][-1]
    formatted = f"## Agent Response\n\n{last.content}"
    return {"messages": [AIMessage(content=formatted)]}

# Register all nodes:
graph = StateGraph(AgentState)
graph.add_node("llm", call_llm)
graph.add_node("tools", call_tools)
graph.add_node("format", format_output)

```

10. Edges — Connecting Nodes

```

from langgraph.graph import StateGraph, END

graph = StateGraph(AgentState)
graph.add_node("a", node_a)
graph.add_node("b", node_b)
graph.add_node("c", node_c)

# ■■ Entry point ■■
graph.set_entry_point("a")      # execution starts here

# ■■ Simple (unconditional) edges ■■
graph.add_edge("a", "b")        # after a → always go to b
graph.add_edge("b", "c")        # after b → always go to c

# ■■ END edge – finish execution ■■
graph.add_edge("c", END)        # after c → done

# ■■ Multiple edges from one node ■■
graph.add_edge("a", "b")        # INVALID for StateGraph – can only have
                                # one unconditional edge from a node
                                # use conditional edges for branching

# ■■ Parallel edges (fan-out) – available in some graph types ■■
# Use conditional edges routing to different nodes instead

```

11. Conditional Edges — Branching Logic

```

from langgraph.graph import END
from langchain_core.messages import AIMessage

# ■■ Routing function – decides next node ■■
def should_use_tools(state: AgentState) -> str:
    last_message = state["messages"][-1]

    # If last message is AIMessage with tool_calls → use tools
    if isinstance(last_message, AIMessage) and last_message.tool_calls:
        return "use_tools"

    # Otherwise → done

```

```

        return "end"

# ■■■ Register conditional edge ■■■
graph.add_conditional_edges(
    "llm",          # FROM this node
    should_use_tools, # CALL this function with current state
    {
        "use_tools": "tools", # if it returns "use_tools" → go to "tools"
        "end": END,          # if it returns "end" → stop
    }
)

# ■■■ More complex routing ■■■
def route(state: AgentState) -> str:
    last = state["messages"][-1]
    if not isinstance(last, AIMessage):
        return "continue"
    if last.tool_calls:
        return "tools"
    if "ERROR" in (last.content or ""):
        return "handle_error"
    if len(state["messages"]) > 20: # too many turns
        return "force_end"
    return "end"

graph.add_conditional_edges("llm", route, {
    "tools": "tools",
    "handle_error": "error_handler",
    "force_end": "summarise",
    "end": END,
    "continue": "llm", # loop back
})

```

12. Compiling and Running Graphs

```

from langgraph.graph import StateGraph, END
from langchain_core.messages import HumanMessage

# Build the graph
graph = StateGraph(AgentState)
graph.add_node("llm", call_llm)
graph.add_node("tools", call_tools)
graph.set_entry_point("llm")
graph.add_conditional_edges("llm", should_use_tools, {"use_tools": "tools", "end": END})
graph.add_edge("tools", "llm")

# Compile
agent = graph.compile()

# ■■■ Run (synchronous) ■■■
result = agent.invoke({
    "messages": [HumanMessage(content="Show me pending ideas")]
})
final_message = result["messages"][-1]
print(final_message.content)

# ■■■ Run (async) ■■■
result = await agent.ainvoke({
    "messages": [HumanMessage(content="Approve idea abc-123")]
})

```

```

}))

# ■■■ Stream output ■■■
async for event in agent.astream({
    "messages": [HumanMessage(content="Tell me about the latest run")]
}):
    # event is a dict: {node_name: state_update}
    for node_name, state_update in event.items():
        print(f"[{node_name}]:", state_update)

# ■■■ Stream just the messages ■■■
async for event in agent.astream_events({
    "messages": [HumanMessage(content="What articles did we scrape today?")]
}, version="v2"):
    if event["event"] == "on_chat_model_stream":
        chunk = event["data"]["chunk"]
        print(chunk.content, end="", flush=True)

```

13. Streaming — Watching the Agent Think

There are 4 levels of streaming in LangGraph:

```

# 1. Stream graph updates (node-level)
async for chunk in agent.astream(initial_state):
    # chunk = {"node_name": {...state_changes...}}
    for node, changes in chunk.items():
        if "messages" in changes:
            last_msg = changes["messages"][-1]
            print(f"[{node}] {type(last_msg).__name__}: {str(last_msg.content)[:80]}")

# 2. Stream token-by-token (sub-graph level) via astream_events
async for event in agent.astream_events(initial_state, version="v2"):
    kind = event["event"]
    if kind == "on_chat_model_stream":
        token = event["data"]["chunk"].content
        if token:
            print(token, end="", flush=True)
    elif kind == "on_tool_start":
        print(f"\nCalling tool: {event['name']} with {event['data']['input']}")
    elif kind == "on_tool_end":
        print(f"\nTool result: {event['data']['output']}")

# 3. Stream over WebSocket (FastAPI integration)
from fastapi import WebSocket

@app.websocket("/ws/chat")
async def ws_chat(websocket: WebSocket):
    await websocket.accept()
    user_message = await websocket.receive_text()

    config = {"configurable": {"thread_id": "ws_session_1"}}
    initial = {"messages": [HumanMessage(content=user_message)]}

    async for event in agent.astream_events(initial, config=config, version="v2"):
        if event["event"] == "on_chat_model_stream":
            token = event["data"]["chunk"].content
            if token:
                await websocket.send_text(token)

```

```
await websocket.send_text("[DONE]")
```

14. Checkpointers — Persistence and Memory

Checkpointers save the graph state after every node execution. This enables: resuming interrupted conversations, human-in-the-loop pauses, and cross-session memory.

```
# ■■■ In-memory checkpointer (lost on restart) ■■■
from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
agent = graph.compile(checkpointer=checkpointer)

# Thread ID separates conversations
config = {"configurable": {"thread_id": "user_123_session_1"}}

# Turn 1
await agent.ainvoke(
    {"messages": [HumanMessage("Show me pending ideas")]},
    config=config,
)

# Turn 2 – agent REMEMBERS turn 1 because of thread_id
await agent.ainvoke(
    {"messages": [HumanMessage("Approve the top one")]},
    config=config,
)

# ■■■ SQLite checkpointer (survives restarts) ■■■
from langgraph.checkpoint.sqlite import SqliteSaver

with SqliteSaver.from_conn_string("checkpoints.db") as checkpointer:
    agent = graph.compile(checkpointer=checkpointer)
    # Now state persists across server restarts

# ■■■ PostgreSQL checkpointer (production) ■■■
from langgraph.checkpoint.postgres import PostgresSaver

with PostgresSaver.from_conn_string(DATABASE_URL) as checkpointer:
    checkpointer.setup() # creates tables – run once
    agent = graph.compile(checkpointer=checkpointer)
```

14.1 Getting State History

```
# Inspect what happened in a thread
config = {"configurable": {"thread_id": "user_123_session_1"}}

# Get current state
state = agent.get_state(config)
print("Current state:", state.values)
print("Next node:", state.next)

# Get full history
for state_snapshot in agent.get_state_history(config):
    print(f"Step {state_snapshot.step}: {state_snapshot.values['messages'][-1].content[:50]}")

# Replay from a specific checkpoint
to_replay = list(agent.get_state_history(config))[3] # 4th checkpoint
```

```

result = await agent.ainvoke(None, config=to_replay.config)

# Update state manually (inject a correction)
agent.update_state(
    config,
    {"messages": [HumanMessage("Actually, approve idea xyz-456 instead")]},
)

```

15. Human-in-the-Loop — Interrupts and Approvals

`interrupt_before` and `interrupt_after` cause the graph to PAUSE at specific nodes. Execution freezes, the state is saved, and you can inspect/modify the state before resuming. This is how you build approval gates.

```

from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver

# Compile with interrupts
checkpointer = MemorySaver()
agent = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=["publish_to_twitter"], # pause BEFORE publishing
    # interrupt_after=["generate_draft"],    # pause AFTER generating
)

config = {"configurable": {"thread_id": "approval_flow_1"}}

# Step 1: Run until the interrupt
for event in agent.stream(
    {"messages": [HumanMessage("Generate and publish a tweet about SEBI")]},
    config=config,
):
    print(event)

# ← Execution PAUSED before publish_to_twitter
# ← State is saved in checkpointer

# Step 2: Inspect what's about to be published
state = agent.get_state(config)
print("Next node:", state.next) # ("publish_to_twitter",)
draft = state.values["messages"][-1].content
print("Draft tweet:", draft)

# Step 3a: Approve – resume with no changes
if user_approves(draft):
    for event in agent.stream(None, config=config): # None = resume
        print(event)

# Step 3b: Reject – modify state then resume
else:
    agent.update_state(
        config,
        {"messages": [HumanMessage("Change the tweet to be more formal")]},
        as_node="llm", # inject the correction as if it came from user
    )
    for event in agent.stream(None, config=config):
        print(event)

```

16. Prebuilt ReAct Agent — Fast Start

LangGraph includes a prebuilt ReAct (Reason + Act) agent that handles the LLM → tool → LLM loop automatically. Great for simple agents without custom routing.

```
from langgraph.prebuilt import create_react_agent
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage
from langgraph.checkpoint.memory import MemorySaver

llm = ChatAnthropic(model="claude-sonnet-4-6")
tools = [get_stock_price, search_ideas, approve_idea]

# Create a prebuilt ReAct agent
checkpoint = MemorySaver()
agent = create_react_agent(
    model=llm,
    tools=tools,
    checkpoint=checkpoint,
    state_modifier="You are a helpful AI that manages an Indian finance content pipeline.",
)

# Run it
config = {"configurable": {"thread_id": "session_1"}}
result = await agent.ainvoke(
    {"messages": [HumanMessage("Show me the top 5 pending ideas for LinkedIn")]},
    config=config,
)
print(result["messages"][-1].content)

# The prebuilt agent automatically:
# 1. Calls the LLM with tools bound
# 2. Detects tool calls in the response
# 3. Executes the tools
# 4. Feeds results back to the LLM
# 5. Repeats until the LLM gives a final answer
```

17. ToolNode — Automatic Tool Execution

```
from langgraph.prebuilt import ToolNode

tools = [get_stock_price, search_ideas, approve_idea, run_research]

# ToolNode automatically executes whatever tools the LLM calls
tool_node = ToolNode(tools)

# Add it as a node in your graph
graph = StateGraph(AgentState)
graph.add_node("llm", call_llm)
graph.add_node("tools", tool_node)  # ← ToolNode handles everything
graph.set_entry_point("llm")
graph.add_conditional_edges("llm", should_use_tools, {"use_tools": "tools", "end": END})
graph.add_edge("tools", "llm")

# ToolNode does the following automatically:
# 1. Reads tool_calls from the last AIMessage
# 2. Finds matching tools by name
# 3. Calls each tool with the provided args
```

```
# 4. Returns ToolMessages with the results
# 5. Handles errors gracefully (returns error message, doesn't crash)

# Custom error handling in ToolNode
from langgraph.prebuilt import ToolNode
from langgraph.utils.runnable import RunnableCallable

def handle_tool_error(state: AgentState) -> dict:
    error = state.get("error")
    last = state["messages"][-1]
    return {"messages": [ToolMessage(
        content=f"Tool failed: {error}",
        tool_call_id=last.tool_calls[0]["id"],
    )]}

tool_node = ToolNode(tools, handle_tool_errors=handle_tool_error)
```

18. Multi-Agent Systems — Supervisor Pattern

In a multi-agent system, one **supervisor** agent orchestrates multiple **worker** agents. Each worker specialises in a domain. The supervisor decides which worker to invoke based on the task.

```
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage

llm = ChatAnthropic(model="claude-sonnet-4-6")

# ■■■ Worker agents ■■■
research_agent = create_react_agent(llm, tools=[scrape_web, extract_article])
scoring_agent = create_react_agent(llm, tools=[score_headline, generate_idea])
writing_agent = create_react_agent(llm, tools=[write_post, check_compliance])

# ■■■ Supervisor agent ■■■
from pydantic import BaseModel
from typing import Literal

class Route(BaseModel):
    next: Literal["research", "scoring", "writing", "FINISH"]

supervisor_llm = llm.with_structured_output(Route)

async def supervisor_node(state: AgentState) -> dict:
    system = """You manage a content pipeline. Route tasks:
- research: for finding new articles
- scoring: for evaluating content ideas
- writing: for creating posts
- FINISH: when the task is complete"""
    route = await supervisor_llm.ainvoke(
        [{"role": "system", "content": system}] + state["messages"]
    )
    return {"next": route.next}

# ■■■ Build the supervisor graph ■■■
from langgraph.graph import StateGraph, END
from typing import Annotated
import operator

class SupervisorState(TypedDict):
```

```

        messages: Annotated[list, operator.add]
        next:      str

graph = StateGraph(SupervisorState)
graph.add_node("supervisor", supervisor_node)
graph.add_node("research",    lambda s: research_agent.invoke(s))
graph.add_node("scoring",     lambda s: scoring_agent.invoke(s))
graph.add_node("writing",     lambda s: writing_agent.invoke(s))

graph.set_entry_point("supervisor")

# All workers route back to supervisor after completion
for worker in ["research", "scoring", "writing"]:
    graph.add_edge(worker, "supervisor")

# Supervisor routes to workers or finishes
graph.add_conditional_edges(
    "supervisor",
    lambda s: s["next"],
    {"research": "research", "scoring": "scoring",
     "writing": "writing", "FINISH": END}
)

```

19. Subgraphs — Modular Architectures

```

# Subgraphs let you use one compiled graph as a node inside another

# ■■■ Define a subgraph ■■■
sub_graph = StateGraph(SubState)
sub_graph.add_node("step1", do_step1)
sub_graph.add_node("step2", do_step2)
sub_graph.set_entry_point("step1")
sub_graph.add_edge("step1", "step2")
sub_graph.add_edge("step2", END)

sub_agent = sub_graph.compile()    # compile the subgraph

# ■■■ Use it as a node in the main graph ■■■
main_graph = StateGraph(MainState)
main_graph.add_node("main_step", main_node)
main_graph.add_node("subprocess", sub_agent)    # subgraph as a node!
main_graph.set_entry_point("main_step")
main_graph.add_edge("main_step", "subprocess")
main_graph.add_edge("subprocess", END)

main_agent = main_graph.compile()

```

20. Error Handling in Graphs

```

# ■■■ Try/except inside nodes ■■■
async def safe_tool_node(state: AgentState) -> dict:
    last = state["messages"][-1]
    results = []
    for call in last.tool_calls:
        try:
            result = await tool_map[call["name"]].ainvoke(call["args"])
        except Exception as e:

```

```

        result = f"Error calling {call['name']}: {str(e)}"
        results.append(ToolMessage(content=str(result), tool_call_id=call["id"]))
    return {"messages": results}

# ■■ Retry logic in a node ■■
import asyncio

async def robust_llm_node(state: AgentState) -> dict:
    for attempt in range(3):
        try:
            response = await llm_with_tools.ainvoke(state["messages"])
            return {"messages": [response]}
        except Exception as e:
            if attempt == 2:
                raise
            await asyncio.sleep(2 ** attempt) # 1s, 2s, 4s

# ■■ Fallback node ■■
def error_handler_node(state: AgentState) -> dict:
    return {"messages": [AIMessage(
        content="I encountered an error. Please try again or rephrase your request."
    )]}

graph.add_node("error_handler", error_handler_node)

def route_on_error(state: AgentState) -> str:
    last = state["messages"][-1]
    if isinstance(last, AIMessage) and "Error" in (last.content or ""):
        return "handle_error"
    return "continue"

graph.add_conditional_edges("llm", route_on_error,
    {"handle_error": "error_handler", "continue": "tools"})

```

21. LangGraph + FastAPI — Full Integration

```

# app/agents/orchestrator/agent.py
from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode
from langgraph.checkpoint.memory import MemorySaver
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import SystemMessage

from app.agents.orchestrator.tools import make_tools

def create_orchestrator(supabase, arq_pool, settings) -> tuple:
    llm = ChatAnthropic(
        model=settings.claude_model_heavy,
        api_key=settings.anthropic_api_key,
    )

    tools = make_tools(
        supabase=supabase,
        arq_pool=arq_pool,
        tavily_api_key=settings.tavily_api_key,
        anthropic_api_key=settings.anthropic_api_key,
    )
    llm_with_tools = llm.bind_tools(tools)

```



```

        elif event["event"] == "on_tool_start":
            await websocket.send_text(
                f"\n[Using tool: {event['name']}]...\n"
            )

        await websocket.send_text("\n[DONE]")

    except WebSocketDisconnect:
        pass

```

22. ContentAutomation2 — Orchestrator Walkthrough

The orchestrator is the brain of the system. It sits at the intersection of every pipeline stage and gives the human operator a natural language interface to control everything.

```

# What happens when a user says:
# "Show me the top 3 pending LinkedIn ideas and approve the best one"

# 1. HumanMessage added to state
# 2. call_llm node invoked → Claude sees the message + all tools
# 3. Claude returns AIMessage with tool_call: get_ideas(status='pending_approval', platform='linkedin', ...)
# 4. ToolNode executes get_ideas → returns formatted list of ideas
# 5. ToolMessage with results added to state
# 6. call_llm invoked again → Claude sees ideas + user intent
# 7. Claude returns AIMessage with tool_call: approve_idea(idea_id='abc-123')
# 8. ToolNode executes approve_idea → updates DB
# 9. ToolMessage with success message added to state
# 10. call_llm invoked → Claude formulates final answer
# 11. Claude returns: "I've approved idea 'abc-123' – LinkedIn post about SEBI rate change"
# 12. No more tool_calls → graph ends
# 13. Final AIMessage streamed token by token to the frontend WebSocket

# Tools available to the orchestrator (30+ tools):
# Pipeline control: trigger_research, trigger_scoring, trigger_creation, login_to_site
# Ideas (Gate 1): get_ideas, approve_idea, reject_idea, bulk_reject_ideas, send_ideas_to_creation
# Drafts (Gate 2): get_drafts, approve_draft, reject_draft
# Brand: add_brand_memory, list_brand_memory
# Subscribers: list_subscribers, add_email_subscriber, remove_email_subscriber
# Analytics: get_analytics_summary, get_topic_performance, get_run_logs
# Browse: get_decision_summaries, get_recent_articles, get_published_posts, list_kb_files
# Sites: add_curated_site, remove_curated_site, list_curated_sites
# Web: search_web, search_and_scrape, generate_post, save_draft

```

23. Testing LangGraph Agents

```

import pytest
from unittest.mock import AsyncMock, MagicMock
from langchain_core.messages import HumanMessage, AIMessage, ToolMessage

# ■■■ Test a single node ■■■
async def test_call_llm_node():
    mock_llm = AsyncMock()
    mock_llm.ainvoke.return_value = AIMessage(content="Here are the ideas...")

    state = {"messages": [HumanMessage("Show ideas")]}
    result = await call_llm(state)
    assert "messages" in result

```

```

    assert isinstance(result["messages"][-1], AIMessage)

# ■■ Test a tool function ■■
async def test_get_ideas_tool():
    mock_supabase = MagicMock()
    mock_supabase.table.return_value.select.return_value = mock_supabase.table.return_value.eq.return_value.limit.return_value.execute.return_value

    tools = make_tools(supabase=mock_supabase, arq_pool=None)
    get_ideas = next(t for t in tools if t.name == "get_ideas")

    result = await get_ideas.ainvoke({"status": "pending_approval"})
    assert "Test angle" in result

# ■■ Integration test with full agent ■■
async def test_full_agent_conversation():
    from langgraph.checkpoint.memory import MemorySaver

    # Use real or mocked tools
    tools = make_tools(supabase=mock_supabase, arq_pool=mock_arq)
    agent = create_react_agent(mock_llm, tools, checkpoint=MemorySaver())

    result = await agent.ainvoke(
        {"messages": [HumanMessage("Show pending ideas")]},
        config={"configurable": {"thread_id": "test_1"}}
    )
    assert len(result["messages"]) > 1
    assert isinstance(result["messages"][-1], AIMessage)

```

24. Debugging & Tracing with LangSmith

```

# LangSmith traces every LLM call, tool call, and state change

# 1. Sign up at smith.langchain.com (free tier available)
# 2. Set environment variables:
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "your_langsmith_api_key"
os.environ["LANGCHAIN_PROJECT"] = "content-automation"

# That's it! All LangGraph/LangChain calls are automatically traced.
# Go to smith.langchain.com to see:
# - Full conversation threads
# - Token usage per call
# - Tool call inputs and outputs
# - Latency breakdown per node
# - Errors with full stack traces

# ■■ Local debugging without LangSmith ■■
# Print state at each step:
async for event in agent.astream(initial_state, config=config):
    print("=" * 50)
    for node, state_delta in event.items():
        print(f"NODE: {node}")
        if "messages" in state_delta:
            for msg in state_delta["messages"]:
                role = type(msg).__name__
                content = str(msg.content)[:100] if msg.content else "[tool call]"
                print(f"  {role}: {content}")

```

```
# ■■ Visualise the graph ■■  
from IPython.display import Image  
img = agent.get_graph().draw_mermaid_png()  
with open("agent_graph.png", "wb") as f:  
    f.write(img)  
# Or in Jupyter: Image(agent.get_graph().draw_mermaid_png())
```